

Semesterprüfung HS24/25
Studienstandort Rapperswil**NAME:** _____ **VORNAME:** _____**MATRIKEL-Nr.:** _____**STUDIENGANG:** _____

Modulcode:	M_WE1
Modulbezeichnung:	WEB ENGINEERING 1
Modulowner:	INFORMATIK
Modulverantwortung:	Frieder Loch
Prüfungsverantwortliche/r:	Frieder Loch
Datum/Zeit:	Donnerstag, 23.01.2025, 14:50 bis 16:50 Uhr

erreichte / max. Punktzahl: _____

*(Beispiel: erreichte / max. Punktzahl: 45 / 60)***Prüfungsnote:** _____**Korrektur:** _____*(Name/Unterschrift)*

Prüfungsinformationen

MIT LÖSUNG**Zeit:** 120 Minuten**Erlaubtes Hilfsmittel:** 3 Blätter A4 (ein- oder doppelseitig beschrieben)

1. Legen Sie Ihre Legitimationskarte auf das Pult.
2. Mobiltelefone und andere elektronische Geräte ausschalten.
3. Tragen Sie Ihren Namen auf diesem Deckblatt ein.
4. Kontrollieren Sie, ob Sie alle Aufgabenblätter erhalten haben.
5. Streichen Sie ungültige Lösungsteile deutlich durch. Mehrfachlösungen sind falsch.
6. Zusatzblätter erhalten Sie bei Bedarf vom Prüfer. Verwenden sie kein eigenes Papier.
7. Geben Sie am Prüfungsende diese Lösung zusammen mit eventuellen Zusatzblättern ab.
8. Vorzeitige Abgabe bis maximal 15 Minuten vor Ende der Prüfung.
9. Bleiben Sie am Prüfungsende sitzen, bis alle Prüfungen eingesammelt wurden.
10. Der Weissraum lässt keine Rückschlüsse auf die Länge der erwarteten Antwort zu.
11. Beschriften Sie lose Blätter mit Ihrem Namen.
12. Schreiben Sie Ihre Lösung möglichst nicht mit Bleistift.
13. Verwenden Sie keinen roten Kugelschreiber.
14. Treffen Sie, falls notwendig, plausible Annahmen und vermerken Sie diese konkret.
15. Diese Prüfung hat einen Anhang mit nützlichen Informationen. Dieser kann abgetrennt werden.
16. Verstösse gegen Cleancode-Richtlinien können zum Punktabzug führen.

Eine erfolgreiche Prüfung wünschen
Frieder Loch und Michael Gfeller

#	Aufgabe	Punkte	Erreicht
1	HTML	18	
2	CSS	11	
3	DOM	10	
4	Usability und Barrierefreiheit	12	
4	JavaScript	20	
5	Promises	11	
6	HTTP und REST	8	
	Total	90	

1 HTML (18P)

- a) HTML wird leider oft als Programmiersprache bezeichnet. Warum ist das falsch? Machen Sie in Ihrer Erklärung insbesondere einen Vergleich mit echten Programmiersprachen wie z.B. JavaScript. (2P)

HTML ist eine Auszeichnungssprache. In Gegensatz zu Sprachen wie z.B. JavaScript beschreibt sie **nicht das Verhalten** eines Programms, **sondern die Struktur** eines Dokuments/Webseite.

- b) Ihr Peer-Review Partner:in hat Ihnen den nachfolgenden HTML-Code abgegeben. Analysieren Sie den Code und bestimmen Sie **3 syntaktische** und **3 semantische** Fehler. Geben Sie anschliessend konkretes Feedback, wie diese Probleme behoben werden können. Tragen Sie Ihre Lösungen ausschliesslich in die Tabelle auf der nächsten Seite ein. (12P)

Gehen Sie davon aus, dass abgekürzte Werte (...) korrekt sind. Es existieren im Code mehr als 3 Probleme in beiden Kategorien. Die doppelte Nennung des gleichen Problems gibt keine Punkte, auch nicht mit anderen Verbesserungen.

```
1 <!DOCTYPE html>
2 <html lang="de">
3 <head>
4     <meta charset="UTF-8">
5     <title>Creme Puff</title>
6     <b>Die Geschichte der ältesten Katze der Welt</b>
7 </head>
8 <body>
9     <main>
10        <section>
11            <h1>Die Geschichte von Creme Puff</h1>
12            <p>Die älteste Katze der Welt wurde <span class="important">38</span> Jahre alt
13            und hiess "Creme Puff". Mit dieser Leistung schaffte es Creme Puff in das
14            Guinness-Buch der Rekorde.</p>
15            Ein Bild von Creme Puff</img>
16        </section>
17        <section>
18            <h1>Allgemeine Informationen:</h1>
19            <ul>
20                <li>Geburtstag: 3. August 1967</li>
21                <li>Todestag: 6. August 2005</li>
22                <li>Alter: 38 Jahre</li>
23            </ul>
24        </main>
25        <aside>
26            <h6>Entdecke weitere Themen</h6>
27            <div>
28                <ul>
29                    <li><a href="..." title="Warum Katzen so gute Haustiere sind"/></li>
30                    <li><a href="..." title="Haftet meine Katze für kaputte Gegenstände?"/></li>
31                </ul>
```

```
32     </div>
33   </aside>
34 </body>
35 </html>
```

Syntaktische Fehler

Zeile	Antwort
6	Problem: Invalides Element im <head> Element. Behebung / Verbesserung: Element durch passendes <meta> Element ersetzen. (Entfernen und Verschieben vom Element ist zwar möglich, aber verändert die Idee hinter dem Text.)
15	Problem: Invalidier Closing-Tag bei Void-Element Behebung / Verbesserung: Closing-Tag entfernen. Text in alt-Attribute schreiben.
23/24	Problem: Fehlender Closing-Tag </section> bei geöffnetem <section> Element. Behebung / Verbesserung: Closing-Tag </section> hinzufügen.

Weitere Lösungen: Zeile 29/30: Invalide Verwendung des <a>-Elements (Fehlender Closing-Tag und Inhalt).

Bewertung: Problem 1P, Lösung 1P, Falsche Kategorie -1P

Semantische Fehler

Zeile	Antwort
12	Problem: Das -Element mit der Klasse «important» ist aus semantischer Sicht nicht aussagekräftig. Behebung / Verbesserung: Stattdessen Elemente wie oder verwenden.

15	Problem: Das <code></code>-Element hat kein passendes <code>alt</code>-Attribut. (Hinweis: Neuerdings sind fehlende <code>alt</code>-Attribute auch Syntaxfehler. Das Problem wird daher in beiden Kategorien akzeptiert, aber nur einmal.) Behebung / Verbesserung: Sinnvolles <code>alt</code>-Attribut hinzufügen.
18	Problem: Doppelte Verwendung des <code><h1></code>-Elements. Behebung / Verbesserung: Zweites <code><h1></code>-Element durch <code><h2></code>, <code><h3></code>, etc. ersetzen.

Weitere Lösungen: Zeile 19: `<ul>`-Element kann durch besser passendes `<dl>`-Element ersetzt werden. Zeile: 27: `<div>`-Element kann durch aussagekräftigeres `<nav>`-Element ersetzt werden.

Bewertung: Problem 1P, Lösung 1P, Falsche Kategorie -1P

- c) Ihr Peer-Review Partner:in hat für das Bild von Creme Puff (rechts) den Alternativtext «Ein Bild von Creme Puff» vorgesehen. Was ist das Problem mit diesem Alternativtext in Bezug auf die Barrierefreiheit (Accessibility)? (2P)



Der Alternativtext ist nur wenig aussagekräftig, da er weder das Bild ausreichend beschreibt noch den Kontext des Textes berücksichtigt. Aus diesem Grund ist der Text für Menschen mit Sehbehinderungen oder schlechtem Internetzugang nicht sehr hilfreich.

Bessere Alternative (nicht erwartet): «Creme Puff ist eine Katze unbekannter Rasse mit braun/weissem Fell.»

- d) Trotz der von Ihnen gefundenen Syntaxfehler wird die Webseite aus Teilaufgabe b) im Browser grösstenteils korrekt dargestellt. Warum ist das so und warum ist das aus der Sicht der Browserbenutzer sinnvoll? (2P)

Im Gegensatz zu den Compilern in anderen Programmiersprachen versuchen die Browser, die geladenen Webseiten bestmöglich darzustellen. Das macht Sinn, weil es einem Browserbenutzer grundsätzlich egal ist, ob eine Website syntaktisch korrekt programmiert wurde, solange sie vernünftig genutzt werden kann.

(Stellen Sie sich einen Browser vor, der bei jeder zweiten Webseite eine Fehlermeldung wegen eines fehlenden `</div>`-Tags anzeigt.)

2 CSS (11P)

In den nachfolgenden Aufgaben sollen Sie verschiedene CSS-Fragen beantworten. Als Grundlage dient das folgende HTML-Dokument. Gehen Sie davon aus, dass der CSS-Code korrekt eingebunden ist und dass die Teilaufgaben unabhängig voneinander sind.

```

1  <main class="one">
2      <section>
3          <h1>Katzen</h1>
4          <p>Eine Katze ist eine Art Hund, die ein wenig seltsam bellt.</p>
5      </section>
6      <section id="three">
7          <h2>Tagesablauf</h2>
8          <p>Tagesablauf einer gewöhnlichen Katze:</p>
9          <ul>
10             <li class="four">11h Schlafen</li>
11             <li>1h Selbstreinigung</li>
12             <li>1h Essen</li>
13             <li>11h Schlafen (erneut)</li>
14         </ul>
15     </section>
16 </main>

```

- a) Schreiben Sie jeweils einen Selektor mit den entsprechenden Deklarationen, so dass die nachfolgenden Anforderungen erfüllt sind. (5P)

Anforderung	Antwort
Die primäre Überschrift (Zeile 3) soll: - Eine Schriftgrösse von 24px haben. - Einen schwarzen, durchgehenden Rand unten von 2px Breite besitzen. - Einen unteren Abstand von 6px zwischen Text und Rand besitzen. (2P)	<pre>h1 { font-size: 24px; border-bottom: 2px solid black; padding-bottom: 6px; }</pre> Selektor: 0.5P Pro Deklaration: 0.5P
Alle Listenelemente sollen beim Darüberfahren mit der Maus eine grüne Textfarbe erhalten. (1P)	<pre>li:hover { color: green; }</pre> Selektor: 0.5P Pro Deklaration: 0.5P
Alle direkten Kinder des Abschnitts mit der ID «three» (Zeile 6) sollen einen Aussenabstand von 5px und einen grauen Hintergrund besitzen. (2P)	<pre>#three > * { margin: 5px; background: gray; }</pre> Selektor: 1P Pro Deklaration: 0.5P

- b) Ausgehend von dem nachfolgenden CSS, welche Farbe hat das erste Listenelement (Zeile 10)? Begründen Sie Ihre Farbwahl. (2P)

CSS	Antwort
<pre>main ul li { color: red; } #three ul .four { color: green; } #three li:first-child { color: blue; } main.one .four { color: yellow; }</pre>	Farbe: Blau (0.5P) Begründung: Der 2. und 3. Selektor haben beide die höchste Spezifität (1,1,1). Da der 3. Selektor weiter unten im Code steht, greift dieser. (1.5P)

- c) Wie lautet der CSS-Selektor, der das direkte Geschwisterelement des ersten Listenelements (also das Element auf Zeile 11) selektiert? (2P)

li:first-child + li (oder .four + li, li.four + li, #three li:nth-child(2), etc.)

- d) Die in den vorherigen Teilaufgaben verwendeten Selektoren (insbesondere Aufgabe b) sind zwar syntaktisch korrekt, aber aus Sicht von Clean-Code mangelhaft. Welche Empfehlung würden Sie einem Peer-Review Partner:in geben, um die Selektoren verständlicher zu machen? Geben Sie neben einer Erklärung auch ein konkretes Beispiel zur Verbesserung an. (2P)

Anstatt komplexe Selektoren sollte man, wenn möglich, semantisch sinnvolle Klassennamen verwenden. Im besten Fall sind diese an verschiedenen Stellen auf der Webseite anwendbar.

z.B. Könnte man anstatt den Selektoren in Aufgabe «b» dem ersten -Element die Klasse «highlight-blue» geben, oder dem -Element die Klasse «highlight-first-blue».

3 DOM (10P)

Ihr Peer-Review Partner:in hat eine Webseite entwickelt, auf der die beliebtesten Katzen der OST-Studierenden angezeigt werden. Schauen Sie sich den Code an und beantworten Sie die folgenden Fragen.

```
1 <div id="cat-container">
2   
3   
4   
5 </div>
6 <p>Klick auf eine Katze, um loszulegen.</p>
7
8 <script>
9   const textContainer = document.querySelector("p")
10  document
11    .querySelectorAll("img")
12    .forEach(element =>
13      element.addEventListener("click",
14        (event) => {
15          textContainer.innerText = event.target.alt
16        }
17      )
18    )
19 </script>
```

a) Was macht der JavaScript-Code (Zeilen 9–18)? (2P)

Dieser Code registriert auf allen `<img>`-Elementen einen `EventListener`, so dass beim Anklicken des Bildes der entsprechende Alternativtext im ersten `<p>`-Element angezeigt wird.

b) Ihr Peer-Review Partner:in möchte es den Studierenden ermöglichen, ihre eigenen Katzenbilder hinzuzufügen. Das entsprechende Formular wurde bereits implementiert.
(Fortsetzung der Aufgabe auf der nächsten Seite)

```
1 <form>
2   <label>
3     Link zum Katzenbild
4     <input type="text" id="src">
5   </label>
6   <label>
7     Alternativtext
8     <input type="text" id="alt">
9   </label>
10  <button>Absenden</button>
11 </form>
```

Ergänzen Sie den nachstehenden Code so, dass beim Absenden des Formulars die entsprechenden Benutzereingaben ausgelesen werden und mit diesen Werten ein neues Bild am Ende des «cat-container» eingefügt wird. Stellen Sie sicher, dass die Webseite beim Absenden des Formulars nicht neu geladen wird. (4P)

```
<script>
const catContainer = document.querySelector("#cat-container");
document
  .querySelector("form")
  .addEventListener("submit", (event) => {

    })
</script>
```

`event.preventDefault()` (1P)

`const src = document.querySelector("#src").value` (0.5P)

`const alt = document.querySelector("#alt").value` (0.5P)

`catContainer.insertAdjacentHTML("beforeend", ``)` (2P)

- c) Nach der erfolgreichen Implementierung des Bildeinfügens stellen Sie fest, dass die Funktionalität des JavaScript-Codes von Teilaufgabe a) für die dynamisch hinzugefügten Bilder nicht funktioniert. Warum ist das so? (1P)

- d) Das Problem mit den dynamischen Bildern aus Teilaufgabe c) kann mit Event-Bubbling gelöst werden. Schreiben Sie den ursprünglichen Code der Aufgabenstellung (Zeilen 8–19) so um, dass Event-Bubbling verwendet wird. (3P)

```
<script>
const textContainer = document.querySelector("p")
document

    (event) => {

        textContainer.innerText = event.target.alt
    }
```

```
}
```

```
</script>
```

```
document
```

```
  .querySelector("#cat-container") (1P)
```

```
  .addEventListener("click", (1P)
```

```
    (event) => {
```

```
      if (event.target.tagName === "IMG") { (0.5P "if" vorhanden, 0.5P "if" korrekt)
```

```
        textContainer.innerText = event.target.alt;
```

```
      }
```

```
    }
```

```
  );
```

4 Usability und Barrierefreiheit (12P)

- a) Kreuzen Sie im folgenden Abschnitt an, ob die Aussagen wahr (W) oder falsch (F) sind. (6P)

Aussage	W	F
Bei der «Usability» geht es darum, Webseiten so zu entwickeln, dass sie für die Benutzer so verständlich und angenehm wie möglich sind.	X	
Sich bewegende Elemente (Pop-ups, Slide-ins, usw.) werden sowohl im zentralen als auch im peripheren Bereich gut wahrgenommen.	X	
Farbcodierungen eignen sich hinsichtlich der Barrierefreiheit gut für die Darstellung von Statusinformationen, da sie die Sichtbarkeit des Systemstatus gewährleisten.		X
Das Gesetz der Nähe besagt, dass man unterschiedliche Elemente auch dann noch leicht voneinander unterscheiden kann, wenn sie nahe beieinander liegen.		X
In einem Hardware-Shop für IT-Experten können durchaus andere Begriffe und Informationen dargestellt werden als in einem Hardware-Shop für Laien.	X	
Möglichkeiten zum Abbrechen oder Zurücksetzen von bestimmten Aktionen sollen vermieden werden, da dies die Benutzer verwirren kann.		X

- b) Ihr Peer-Review Partner:in hat Ihnen den nachstehenden Text für Ihre Webseite zugeschickt. Wie Sie sehen können, wurde der Strukturierung des Textes keine Aufmerksamkeit geschenkt. Nennen und erläutern Sie **einen** konkreten Ansatz, wie der Text strukturiert werden kann, um seine Lesbarkeit zu verbessern. (2P)

Für die Beantwortung dieser Aufgabe wird **kein** Code erwartet.

Die Abenteuer von Mieze und dem Käsetoast
Eines Tages fand die Katze Mieze einen magischen Käsetoast in der Küche. "Dieser Käsetoast spricht!", rief Mieze erstaunt.
Der sprechende Käsetoast
Der Käsetoast sagte: "Iss mich nicht, ich erfülle dir drei Wünsche!" Mieze überlegte kurz und wünschte sich mehr Käsetoast, ein gemütliches Kissen und eine endlose Spielzeugmaus.

Antwort:

Man könnte bei diesem Text z.B. die Überschriften herausheben, indem man diese in einer grösseren/breiteren Schriftart darstellt. Andere Möglichkeiten wären u.A. nach Robby Ingebartsen (Woche 13, Folie 21) die Verwendung von Abständen, Einrückungen, Symbolen oder Farben.

- e) In der Vorlesung haben Sie verschiedene Methoden kennengelernt, wie man Benutzer im Bereich der Wahrnehmung sinnvoll durch eine Webseite führen kann. Leider werden diese Methoden nicht nur von aufrichtigen Entwickler:innen wie Ihnen verwendet.

Nennen Sie anhand der nachfolgenden Webseite **zwei** konkrete Beispiele, wie durch den Aufbau und/oder die Gestaltung der Webseite versucht wird, den Benutzer zum Anklicken eines Werbebanners zu bewegen. Beziehen Sie sich dabei konkret auf die in der Vorlesung gezeigten Prinzipien. (4P)

The screenshot shows a software download page for 'ADL Uncompressor'. It features several buttons and text elements that are annotated with arrows and labels:

- Top Section:** A green button labeled 'Download' with a download icon. An arrow points to it from the label '← Werbebanner'.
- Middle Section:** A light orange box containing a 'Download Now' button (blue text, green icon) and a 'Free Download' button (green text, green icon). An arrow points to the 'Download Now' button from the label '← Echter Download'.
- Bottom Section:** A large green button labeled 'Free Download' with a download icon. An arrow points to it from the label '← Werbebanner'.

Other visible text includes: 'Thank you for downloading', 'is a freeware supporting many like', 'Add to My WatchList', 'Share: [social media icons]', 'Q&A Board', 'Rating: ★★★★★', '...Recommended Download', 'License: Free', 'Download Size: 625 KB', 'Supported OS: Windows XP/Vista/7', and 'Software: [icon]'.

Antwort:

Verschiedene Antworten sind möglich. Z.B.:

- Gesetz der Ähnlichkeit: Alle Buttons sehen mehr oder weniger gleich aus, wodurch wir sie nicht mehr so einfach voneinander unterscheiden können.
 - Visuelle Hierarchie: Die Download-Buttons werden bewusst ganz gross dargestellt, währendem die Information zur tatsächlichen Software in 2/3 Fällen ganz klein und am Rand dargestellt wird.
-

5 JavaScript (20P)

5.1 Theorie (5P)

Notieren Sie das Resultat bzw. die Ausgabe des jeweiligen Ausdrucks:

Ausdruck	Resultat
<code>"" ?? "1"</code>	<code>""</code>
<code>false == '1' == 0</code>	<code>True</code>
<code>const a = "aa" const b = "bb" console.log({a, ...b})</code>	<code>{0: 'b', 1: 'b', a: 'aa'}</code>
<code>const items = [1,2,3,4] console.log(items.reduce((a, b) => a + b, 10));</code>	<code>20</code>
<code>'b' + 'a' + + 'a' + 'a'</code>	<code>baNaNa</code>

5.2 Programmieren (5P)

Programmieren Sie die `groupAndCount(items, field)`-Funktion. Die Funktion muss die folgenden Anforderungen erfüllen. Beachten Sie auch die Beispielaufufe.

- Die Funktion erhält eine Liste von Objekten (`items`). Die Objekte werden nach einem bestimmten Feld (`field`) gruppiert. Die Elemente in jeder Gruppe werden gezählt.
- Werte wie `null` / `undefined` werden ignoriert.
- Nutzen Sie passende Array-Methoden.
- Nutzen Sie keine `if else` / `for` / `while` / `switch case`. Verwendung führt zu Punktabzug.

Beispiel Daten:

```
const items = [  
  {item: "Ananas", category: "Fruit"},  
  {item: "Shirt", category: "Clothing"},  
  {item: "Car", category: "Vehicle"},  
  {item: "Pineapple", category: "Fruit"},  
  {item: "Bread", category: null}  
];
```

Beispiel Aufruf:

```
console.log(groupAndCount(items, "category"));  
// { Fruit: 2, Clothing: 1, Vehicle: 1 }  
console.log(groupAndCount(items, "item"));  
// { Ananas: 1, Shirt: 1, Car: 1, Pineapple: 1, Bread: 1 }  
console.log(groupAndCount(items, "nonexistentField"));  
// {}
```

```
function groupAndCount(items, field) {
```

```
}
```

```
function groupAndCount(items, field) {  
  return items  
    .filter(item => item[field] != null) // 2P für den Filter  
    .reduce((acc, item) => { // 3P für die Summe ( 1 für den default ||, 2 für die Summe )  
      acc[item[field]] = (acc[item[field]] || 0) + 1;  
      return acc;  
    }, {});  
};
```

größere Fehler -1...-3P

5.3 Ausgabe mit Begründung (10P)

Folgender Code ist gegeben für die nachfolgenden Unteraufgaben 5.3.1 und 5.3.2:

```
global["#name"] = "Dummy";  
function call(fn){  
    console.log(fn())  
}  
class Person{  
    #name = "Lisa";  
    getName(){  
        return this.#name;  
    }  
}  
const person = new Person()
```

5.3.1 Was ist die Ausgabe von folgendem Ausdruck. Begründen Sie.

```
console.log(person.#name);
```

Ausgabe: _____

Begründung:

5.3.2 Folgender Ausdruck erzeugt nicht wie erwartet «Lisa», sondern was?

```
call(person.getName);
```

Ausgabe: _____

Begründung:

Nennen Sie drei sinnvolle Möglichkeiten damit «Lisa» korrekt ausgegeben wird.

Änderung 1:

Begründung 1:

Änderung 2:

Begründung 2:

Änderung 3:

Begründung 3:

1:
Error / Kein Zugriff auf private Variablen
(undefined / richtige Begründung -> 1P)

2a:
Error / Context geht verloren und die Klasse ist im Strict Modes

2b:
// 2a Lösung:
call(() => person.getName());
// -> Begründung: Context wird mitgegeben
person.getName.bind(person);
// -> Context wird gebunden
// In der Klasse: getName = () => this.#name
// -> Mit Hilfe vom Lambda wird der Context gebunden

Idee: Sichtbarkeit Ändern der Variable gibt keinen Punkt

6 Promises (11P)

6.1 Schreiben Sie die Funktion `echoAsync` (5P)

- Die Funktion soll asynchron mit Promises funktionieren und ein Promise zurückgeben.
- Soll nach 5 Sekunden den Parameter `text` wieder zurückgeben.
- Soll sofort einen Error erzeugen, falls der Text kein string ist.

```
function echoAsync(text) {
```

```
}
```

Beispielaufrufe:

```
echoAsync("hello world")  
  .then(response => console.log(response))  
  .catch(error => console.error(error));  
// -> nach 5 Sekunden -> "hello world"
```

```
echoAsync(42)  
  .then(response => console.log(response))  
  .catch(error => console.error(error));  
// -> Input must be a string
```

```
return new Promise((resolve, reject) => { 1P
  if (typeof text !== 'string') { 1P ( nur falls setTimeout nicht aufgerufen wird )
    reject(new Error("Input must be a string")); 1P
    return;
  }
  setTimeout(() => { 2P
    resolve(text);
  }, 5000);
});
```

falls if () innerhalb vom setTimeout ist -> keine Punkte für Fehler und if
-1..-3 Abzug für grobe Fehler.

6.2 async / await (4P)

- Die Funktion echoAsync soll neu auch mit async / await funktionieren.
- Beschreiben Sie welche Änderungen an der Funktion echoAsync nötig sind.
- Schreiben Sie einen der beiden Beispielaufufe von 6.1 um, damit dieser async / wait nutzt.

Änderungen an echoAsync:

Beispielaufuf mit async / await:

Die function mit async markieren -> 1P

```
try{ 2P für try / catch
  console.log(await echoAsync("hello world")) 1P
}
catch(error){
```

```
console.log(error);  
}
```

6.3 Was gibt folgender Code aus? (2P)

```
Promise.resolve('Success')  
  .then(result => {  
    console.log(result)  
    return Promise.reject('Failed');  
  })  
  .then(() => console.log('Progress'))  
  .catch(err => console.log(err))  
  .then(() => console.log('Done'))
```

Ausgabe:

Success, Failed, Done (alles muss korrekt sein)

7 HTTP & REST (8P)

Die folgenden Aussagen beschreiben Eigenschaften von REST und HTTP.
Kreuzen Sie an, ob die Aussagen wahr (W) oder falsch (F) sind. (4P)

Aussage	W	F
Bei einer HTTP-Response können Daten im Query und im Body übertragen werden.		X
GET-Anfragen sind per definition Read-Only	X	
Bei einem 400er Status Code ist der Client für den Fehler verantwortlich.	X	
Die verwendete HTTP-Version wird vor dem ersten Request bestimmt.		X
HATEOAS beschreibt eine Optimierung im HTTP-Protokoll		X
REST ist standardisiert		X
REST verwendet Links, um Ressourcen zu verlinken	X	
PATCH wird in REST verwendet, um eine Ressource zu überschreiben.		X

7.1 Theorie (4P)

Wann sollte der Status Code 201 verwendet werden?

Was bedeutet es, wenn eine HTTP-Methode idempotent ist?

Die nachfolgende Anfrage gibt alle Bestellungen vom Kunden mit der ID 1 zurück.
GET customers/1/orders

Diese Anfrage verwendet eine Sub-Ressource. Ist dies ein gutes oder schlechtes Beispiel für eine Sub-Ressource. Begründen Sie Ihre Antwort:

☐ Gutes Beispiel ☐ Schlechtes Beispiel

Begründung:

Erfolgreich & neue Ressource wurde erstellt. «nur» created -> 0.5P

Mehrfach ausgeführt -> gleiches Ergebniss

Schlechtes Beispiel; Order auch separat ansteuerbar

Gutes Beispiel: Gute Begründung: 2P; OK Begründung 1P für eine gute Antwort braucht es etwas mit unabhängig / nicht anhängig.

;

8 Anhang

8.1 head

The `<head>` HTML element contains machine-readable information (metadata) about the document.

[Content categories](#)

None.

Permitted content

If the document is an `<iframe>` `srcdoc` document, or if title information is available from a higher level protocol (like the subject line in HTML email), zero or more elements of metadata content.

Otherwise, one or more elements of metadata content where exactly one is a `<title>` element.

Tag omission

The start tag may be omitted if the first thing inside the `<head>` element is an element.

The end tag may be omitted if the first thing following the `<head>` element is not a space character or a comment.

Permitted parents

An `<html>` element, as its first child.

8.2 section

The `<section>` HTML element represents a generic standalone section of a document, which doesn't have a more specific semantic element to represent it.

[Content categories](#)

[Flow content](#), [Sectioning content](#), [palpable content](#).

Permitted content

[Flow content](#).

Tag omission

None, both the starting and ending tag are mandatory.

Permitted parents

Any element that accepts [flow content](#). Note that a `<section>` element must not be a descendant of an `<address>` element.

8.3 img

The `<img>` HTML element embeds an image into the document.

[Content categories](#)

[Flow content](#), [phrasing content](#), [embedded content](#), [palpable content](#). If the element has a `usemap` attribute, it also is a part of the interactive content category.

Permitted content

None; it is a [void element](#).

Tag omission

Must have a start tag and must not have an end tag.

Permitted parents

Any element that accepts embedded content.

8.4 a

The `<a>` HTML element (or anchor element), with its `href` attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address.

Content categories	Flow content , phrasing content , interactive content , palpable content.
Permitted content	Transparent , except that no descendant may be interactive content or an <code><a></code> element, and no descendant may have a specified tabindex attribute.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content , but not other <code><a></code> elements.

8.5 b

The `<b>` HTML element is used to draw the reader's attention to the element's contents, which are not otherwise granted special importance. You should not use `<b>` for styling text or granting importance.

Content categories	Flow content , phrasing content , palpable content.
Permitted content	Phrasing content .
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts phrasing content .

8.6 em

The `<em>` HTML element marks text that has stress emphasis. The `<em>` element can be nested, with each level of nesting indicating a greater degree of emphasis.

Content categories	Flow content , phrasing content , palpable content.
Permitted content	Phrasing content .
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts phrasing content .

8.7 span

The `<span>` HTML element is a generic inline container for phrasing content, which does not inherently represent anything.

Content categories	Flow content , phrasing content .
Permitted content	Phrasing content .
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts phrasing content , or any element that accepts flow content .

8.8 ul

The `<ul>` HTML element represents an unordered list of items, typically rendered as a bulleted list.

Content categories	Flow content , and if the <code></code> element's children include at least one <code></code> element, palpable content .
Permitted content	Zero or more <code></code> , <code><script></code> and <code><template></code> elements.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .

8.9 ol

The `<ol>` HTML element represents an ordered list of items, typically rendered as a numbered list.

Content categories	Flow content , and if the <code></code> element's children include at least one <code></code> element, palpable content .
Permitted content	Zero or more <code></code> , <code><script></code> and <code><template></code> elements.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .

8.10 dl

The `<dl>` HTML element represents a description list. Common uses for this element are to implement a glossary or to display metadata (a list of key-value pairs).

Content categories	Flow content , and if the <code><dl></code> element's children include one name-value group, palpable content.
Permitted content	Either: Zero or more groups each consisting of one or more <code><dt></code> elements followed by one or more <code><dd></code> elements, optionally intermixed with <code><script></code> and <code><template></code> elements. Or: (in WHATWG HTML, W3C HTML 5.2 and later) One or more <code><div></code> elements, optionally intermixed with <code><script></code> and <code><template></code> elements.
Tag omission	None, both the starting and ending tag are mandatory.
Permitted parents	Any element that accepts flow content .